2. *Sorozattal reprezentált típusok*

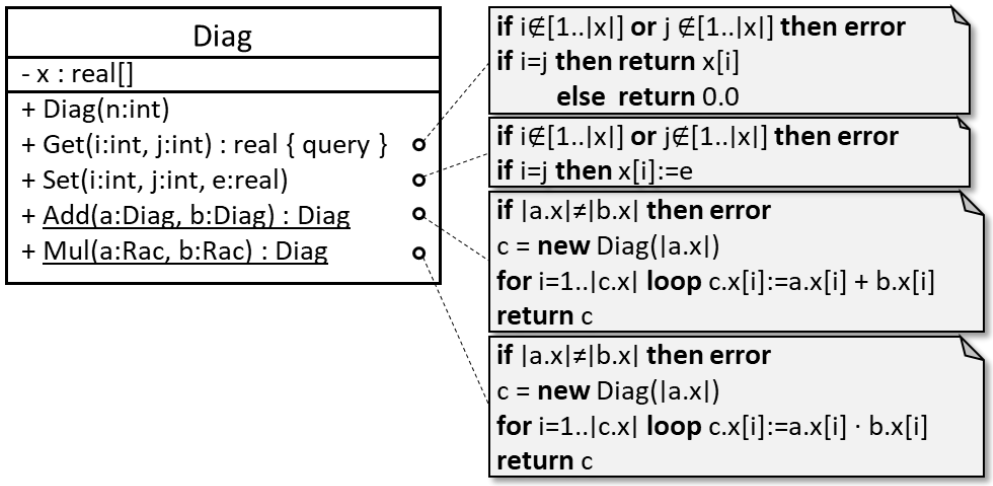
Olyan típusok definiálása, majd osztály diagrammal történő leírása, ahol a reprezentációhoz elemek sorozatára van szükségünk.

1. Diagonális mátrix
   Valósítsuk meg a diagonális mátrix típust (az ilyen mátrixoknak csak a főátlójukban lehetnek nullától eltérő elemek)! Ilyenkor elegendő csak a főátlóbeli elemeket tárolni egy sorozatban. Implementáljuk a mátrix i-edik sorának j-edik elemét lekérdező, illetve megváltoztató műveleteket, valamint két mátrix összegét és szorzatát kiszámoló műveleteket!

   Típusdefiníció:

Diag($\mathbb{R}^{n\times n}$)	e := a[i,j]	(a : Diag($\mathbb{R}^{n\times n}$), i,j : [1..n], e : $\mathbb{R}$)
	a[i,j] := e	(a : Diag($\mathbb{R}^{n\times n}$), i,j : [1..n], e : $\mathbb{R}$) //ha i=j
	c := a + b	(a, b, c : Diag($\mathbb{R}^{n\times n}$))
	c := a · b	(a, b, c : Diag($\mathbb{R}^{n\times n}$))
x:$\mathbb{R}^n$	**if** i=j **then** e := a.x[i] **else** e := 0.0 **endif**	
	if i=j **then** a.x[i] := e **else** error **endif**	
	∀i∈[1..n]: c.x[i]:=a.x[i] + b.x[i]	
	∀i∈[1..n]: c.x[i]:=a.x[i] · b.x[i]	[1.kvíz]

   Osztály diagram:



2. Alsóháromszög mátrix

Valósítsuk meg az alsó háromszög mátrix típust (a mátrixok a főátlójuk felett csak nullát tartalmaznak)! Ilyenkor elegendő csak a főátló és az alatti elemeket reprezentálni egy sorozatban. Implementáljuk a mátrix *i*-edik sorának *j*-edik elemét *lekérdező, illetve megváltoztató* műveletet, valamint két mátrix összegét és szorzatát!

Típusdefiníció:

7 x 7 -es alsó háromszög mátrix

	1	2	3	4	5	6	7
1	1						
2	2	3					
3	4	5	6				
4	7	8	9	10			
5	11	12	13	14	15		
6	16	17	18	19	20	21	
7	22	23	24	25	26	27	28

A sorszámok mutatják, hogy hányadik lesz a mátrix elem a tömbben.

A mátrix elhelyezése sorfolytonosan egy egydimenziós tömbben

(a biztosan nulla értékű elemeket felesleges tárolni)

1	[1,1]
2	[2,1]
3	[2,2]
4	[3,1]
5	[3,2]
6	[3,3]
27	[7,6]
28	[7,7]
29	0

a nulla értéket is tárolhatjuk egy példányban

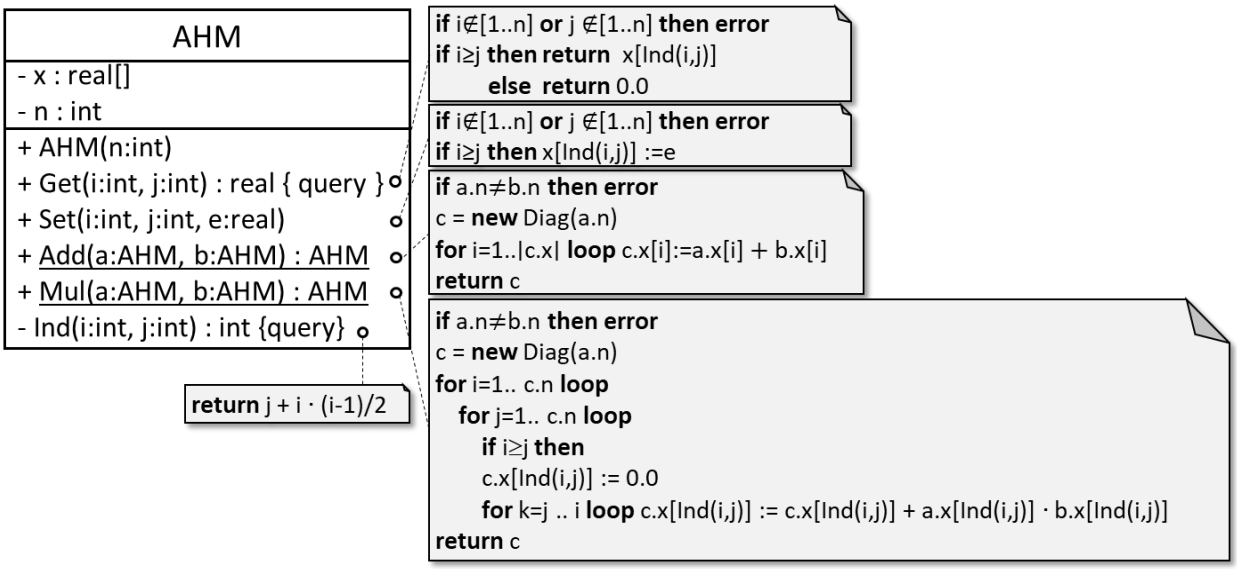
Az index függvény rendeli hozzá a mátrix elem idexeihez a tárolásnak megfelelő indexet:

$$ind(i,j) = j + \sum_{k=1}^{i-1} k = j + \frac{i(i-1)}{2} \quad , ha\ 1 \le j \le i \le n$$

[2.kvíz]

AHM($\mathbb{R}^{n\times n}$)	e := a[i,j] (a : AHM($\mathbb{R}^{n\times n}$), i,j : [1..n], e : $\mathbb{R}$)
	a[i,j] := e (a : AHM($\mathbb{R}^{n\times n}$), i,j : [1..n], e : $\mathbb{R}$) //ha i ≥ j
	c := a + b (a, b, c : AHM($\mathbb{R}^{n\times n}$)) // azonos n-re
	c := a · b (a, b, c : AHM($\mathbb{R}^{n\times n}$)) // azonos n-re
x:$\mathbb{R}^{n(n+1)/2}$	**if** i≥j **then** e := a.x[ind(i,j)] **else** e := 0.0
	if i≥j **then** a.x[ind(i,j)] := e **else** **error**
	∀i∈[1..n(n+1)/2]: c.x[i] := a.x[i]+b.x[i]
	∀i,j∈[1..n]: [3.kvíz] **if** i≥j **then** c.x[ind(i,j)]:= $\sum_{k=j}^{i}$ a.x[ind(i,k)] · b.x[ind(k,j)]

Osztály diagram:



3. Zsák

Valósítsuk meg az egész számokat tartalmazó zsák típusát úgy, hogy nincs felső korlát a zsákba bekerülő egész számok számára. A szokásos (üres-e, betesz, kivesz, hányszor van benn egy szám) műveletek mellett szükségünk lesz a leggyakoribb elemet lekérdező műveletre is.

Az előadáson bemutatott változattal szemben, most egy olyan zsák típust kellene definiálni, ahol nincs felső korlát a zsákba bekerülő természetes számokra. (Sőt, a zsák elemeinek típusa lehet akármilyen típus, csak lehessen annak értékeit sorba rendezni.

<table>
<tr>
<td>Bag
azon zsákok halmaza, amelyek elemei (E) rendezhetőek</td>
<td>b := SetEmpty(b) b : Bag // kiüríti a zsákot
l := Empty(b) b : Bag, l : 𝕃 // üres-e zsák
c:= Multipl(b, e) b : Bag, e : E, c : ℕ // elem multiplicitása
b := Inser(b, e) b : Bag, e : E // elemet tesz be
b:= Remove(b, e) b : Bag, e : E // elemet vesz ki
m:= Max(b) b : Bag, m : E // leggyakoribb elem</td>
</tr>
<tr>
<td>[4.kvíz]
seq : Element*
az elemeket a tartalmuk (data) szerint rendezetten tároló sorozat, ahol
Element =
rec(data: E, count: ℕ)
maxind : ℕ
a seq sorozat legnagyobb count értékű elemének indexe</td>
<td>
b := SetEmpty(b) b : Bag
<table><tr><td>seq := <></td></tr></table>
l := Empty(b) b : Bag, l : 𝕃
<table><tr><td>l := |seq|=0</td></tr></table>
c := Multipl(b, e) b : Bag, e : E, c : ℕ
<table><tr><td>l, ind := logSearch(seq, e)
c :=seq[ind].count</td></tr></table>
m := Max(b) b : Bag, m : E
<table><tr><td colspan="2">|seq| > 0</td></tr><tr><td>m := seq[maxind].data</td><td>hiba</td></tr></table>
b := Insert(b,e) b : Bag, e : E
<table>
<tr><td colspan="5">l, ind := logSearch(seq, e)</td></tr>
<tr><td colspan="5">l</td></tr>
<tr><td colspan="2">++seq[ind].count</td><td colspan="3">seq := seq[1..ind-1] ⊕ <(e,1)> ⊕ seq[ind..|seq|]</td></tr>
<tr><td>seq[ind].count > seq[maxind].count</td><td></td><td>|seq|=1</td><td>maxind≥ind</td><td>else</td></tr>
<tr><td>maxind := ind</td><td>–</td><td>maxind := 1</td><td>++maxind</td><td>–</td></tr>
</table>
</td>
</tr>
</table>

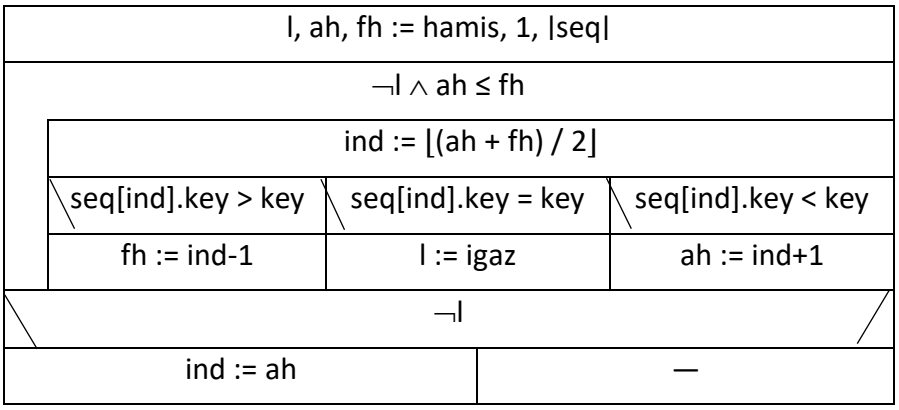
Elem szerinti keresés [elég a logaritmikus keresés lényegét, algoritmusát bemutatni]

A = (seq : Element*, e : E, l : $\mathbb{L}$, ind : $\mathbb{N}$)

Ef = (seq=seq_0 $\wedge$ e=e_0 $\wedge$ $\forall i \in [1 .. |seq|-1]$: seq[i].data < seq[i+1].data)

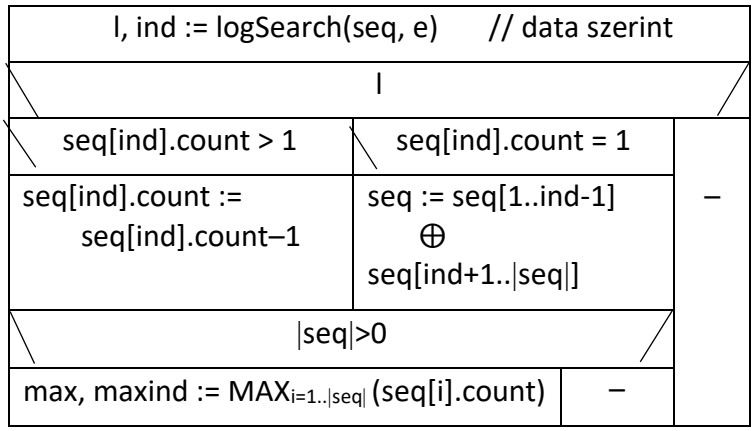
Uf=(Ef $\wedge$ l = $\exists i \in [1 .. |seq|]$: seq[i].data = e $\wedge$
(l $\rightarrow$ ind$\in$[1 .. |seq|] $\wedge$ seq[ind].data = e) $\wedge$
($\neg$l $\rightarrow$ $\forall i \in$ [1..ind-1]: seq[i].data < e $\wedge$ $\forall i \in$[ind.. |seq|] : seq[i].data > e))

l, ind := logSearch (seq, e)

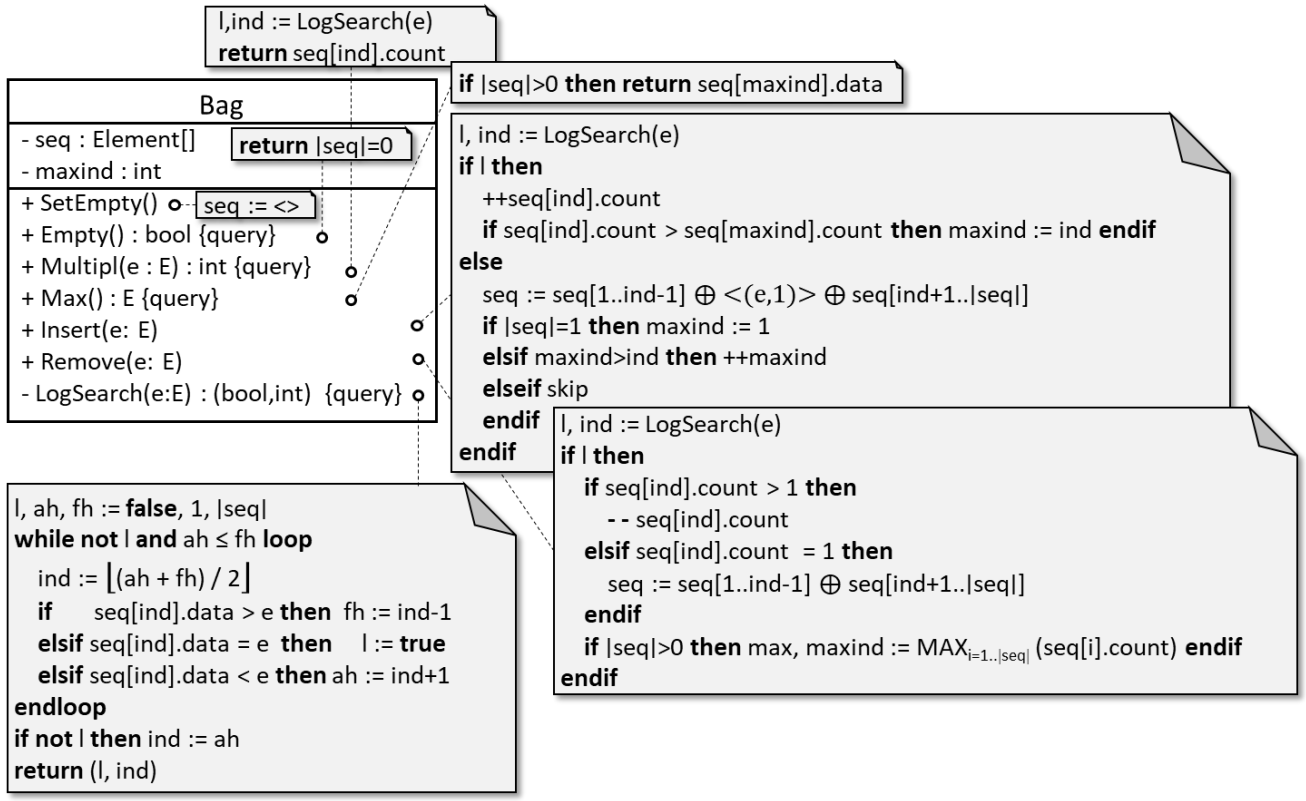


[5.kvíz]

b := Remove(b,e) b : Bag, e : E



Osztály:


l,ind := LogSearch(e)
return seq[ind].count
Bag
- seq : Element[]
- maxind : int
return |seq|=0
+ SetEmpty()
seq := <>
+ Empty() : bool {query}
+ Multipl(e : E) : int {query}
+ Max() : E {query}
+ Insert(e: E)
+ Remove(e: E)
- LogSearch(e:E) : (bool,int) {query}
if |seq|>0 then return seq[maxind].data
l, ind := LogSearch(e)
if l then
++seq[ind].count
if seq[ind].count > seq[maxind].count then maxind := ind endif
else
seq := seq[1..ind-1] ⊕ <(e,1)> ⊕ seq[ind+1..|seq|]
if |seq|=1 then maxind := 1
elsif maxind>ind then ++maxind
elseif skip
endif
endif
l, ah, fh := false, 1, |seq|
while not l and ah ≤ fh loop
ind := ⌊(ah + fh) / 2⌋
if seq[ind].data > e then fh := ind-1
elsif seq[ind].data = e then l := true
elsif seq[ind].data < e then ah := ind+1
endloop
if not l then ind := ah
return (l, ind)
l, ind := LogSearch(e)
if l then
if seq[ind].count > 1 then
- - seq[ind].count
elsif seq[ind].count = 1 then
seq := seq[1..ind-1] ⊕ seq[ind+1..|seq|]
endif
if |seq|>0 then max, maxind := MAX i=1..|seq| (seq[i].count) endif
endif